OOP, Collections, Exceptions & Generics

The four pillars, the framework, and the error handling they'll drill you on

10 topics · 82 flashcards · 48 quiz questions · 37 code examples

How to use this in 4 days

1. **Answer the quiz before you read.** Getting one wrong makes the answer stick; reading twice does not.
2. **Say it out loud.** Every topic ends with a scripted interview answer in the orange box. Reading it silently is not studying.
3. **Type the code.** Your hands remember what your eyes skim past.
4. **Days 1 and 2 are non-negotiable.** OOP, Collections and exceptions are where fresher rounds are won.
5. **Rehearse the project pitch out loud, three times.** It is the highest-leverage question in a fresher interview.

Day 2: OOP, Collections, Exceptions & Generics

The four pillars, the framework, and the error handling they'll drill you on

10 topics · 82 flashcards · 48 quiz questions · ~6.6 hours

> **Goal.** This is the day that decides the interview. OOP and Collections are where fresher rounds live or die — aim to explain every heading here out loud, in your own words, without notes.

Schedule

- Slot 1 (2h): OOP pillars, classes, constructors, this/super
- Slot 2 (2h): Overloading vs overriding, abstraction, abstract vs interface
- Slot 3 (2h): Object methods + equals/hashCode contract + exceptions
- Slot 4 (2h): Collections — List, Set, Map, HashMap internals
- Slot 5 (2h): Iterators, Comparable/Comparator, Generics + wildcards

1. The 4 Pillars of OOP (with real examples) Must-know 35 min

The opener is almost always "what are the four pillars of OOP?" — and the follow-up is "give me a real example of each." Textbook definitions alone sound rehearsed; a one-line concrete example each sounds like someone who's written code.

Key points

- **Encapsulation** — bundle data + behaviour, hide internals behind accessors. `private` fields with getters/setters, so you can add validation later without breaking callers.
- **Inheritance** — `extends`, IS-A relationship, code reuse. Single inheritance for classes in Java (one superclass), multiple inheritance of *type* via interfaces.
- **Polymorphism** — one interface, many forms. Compile-time (method overloading) and runtime (method overriding + dynamic dispatch).
- **Abstraction** — expose *what* an object does, hide *how*. Achieved with abstract classes and interfaces.
- If they ask for a 5th: **Association/Aggregation/Composition** — "has-a" relationships, with composition being the strong one (child can't exist without parent).

Reference

Pillar	One-liner	Java mechanism	Real example
Encapsulation	Hide the data, expose behaviour	private fields + getters/setters	BankAccount hides `balance`; deposit() validates
Inheritance	Child reuses parent's code	`extends` (IS-A)	SavingsAccount extends BankAccount
Polymorphism	Same call, different behaviour	Overloading (compile) / Overriding (runtime)	List l = new ArrayList<>() — code works for any List
Abstraction	What, not how	abstract class / interface	PaymentService interface; StripePayment implements it

Code

All four pillars in one small design

```
// ABSTRACTION: what a payment does, not how
interface PaymentService {
   boolean pay(double amount);        // implicitly public abstract
}

// INHERITANCE + implementation
abstract class CardPayment implements PaymentService {
   protected String cardNumber;       // protected: visible to subclasses
   public boolean pay(double amount) {
       if (!validate()) return false;   // shared behaviour
       return doCharge(amount);         // polymorphic: subclass decides how
   }
   private boolean validate() { return cardNumber != null && cardNumber.length() == 16; }
   protected abstract boolean doCharge(double amount);
}

class CreditCardPayment extends CardPayment {
   @Override protected boolean doCharge(double amount) {
       System.out.println("Charging credit card " + amount);
       return true;
   }
}
class DebitCardPayment extends CardPayment {
   @Override protected boolean doCharge(double amount) {
       System.out.println("Charging debit card " + amount);
       return true;
   }
}

// POLYMORPHISM: same call, different behaviour at runtime
PaymentService p = new CreditCardPayment();
p.pay(499.0);                     // -> "Charging credit card 499.0"
p = new DebitCardPayment();
p.pay(499.0);                     // -> "Charging debit card 499.0"

// ENCAPSULATION
class BankAccount {
   private double balance;                        // hidden
   public double getBalance() { return balance; }
   public void deposit(double amt) {              // behaviour controls state
       if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
       balance += amt;
   }
}
```

Output:
```
Charging credit card 499.0
Charging debit card 499.0
```

Composition over inheritance — the design answer that impresses

```
// Fragile: inheritance locks Engine behaviour into the class hierarchy
class Car extends Engine { }         // Car IS-A Engine? No. Wrong relationship.

// Better: composition - Car HAS-A Engine, swap it at will
class Car {
   private Engine engine;                         // has-a
   Car(Engine engine) { this.engine = engine; }  // inject it
   void drive() { engine.start(); }
}
interface Engine { void start(); }
class PetrolEngine implements Engine { public void start() { System.out.println("vroom"); } }
class ElectricEngine implements Engine { public void start() { System.out.println("whirr"); }
   }

new Car(new ElectricEngine()).drive();   // whirr  <- swap without touching Car
```

Output:
```
whirr
```

> "Prefer composition over inheritance" — inheritance breaks encapsulation and couples you to a parent's implementation. Composition + interfaces is what DI/Spring is built on.

SAY IT LIKE THIS IN THE INTERVIEW

"Encapsulation hides state behind behaviour, inheritance reuses code through an IS-A relationship, polymorphism lets the same call take many forms — overloading at compile time, overriding at runtime — and abstraction exposes what an object does via interfaces and abstract classes. In practice I favour composition over inheritance, because it keeps classes decoupled and testable."

Flashcards

Q. Name the 4 pillars of OOP with a one-line example each.
A. Encapsulation (private balance + deposit()), Inheritance (SavingsAccount extends BankAccount), Polymorphism (PaymentService p = new CreditCardPayment()), Abstraction (interface PaymentService).

Q. Why prefer composition over inheritance?
A. Inheritance couples the subclass to the superclass's implementation and breaks encapsulation. Composition (has-a) lets you swap behaviour at runtime and keeps classes independent.

Q. What's the difference between association, aggregation and composition?
A. Association is a loose 'uses-a'. Aggregation is a weak 'has-a' where parts outlive the whole (Car–Wheel). Composition is a strong 'has-a' where parts die with the whole (House–Room).

Q. Does Java support multiple inheritance?
A. Not for classes (one extends only). But a class can implement multiple interfaces, which gives multiple inheritance of TYPE — and since Java 8, of behaviour too via default methods.

Q. Compile-time vs runtime polymorphism?
A. Compile-time = method overloading (decided by the compiler from the argument types). Runtime = method overriding (decided by the JVM from the actual object type).

Q. What problem does encapsulation actually solve?
A. It protects invariants — validation lives in one place (setters/methods), internals can change without breaking callers, and fields can be made read-only.

Quiz

Q1. Which pillar is demonstrated by `List<String> l = new ArrayList<>();`?

[ ] A. Encapsulation
[ ] B. Inheritance
[x] C. Polymorphism (coding to the interface)
[ ] D. Abstraction only

Why: Programming to the interface is polymorphism: the variable's declared type is List, but calls dispatch to ArrayList's implementation. Swap in LinkedList and nothing changes.

Q2. Why is `class Car extends Engine` considered poor design?

[ ] A. Because Java forbids it
[x] B. Because Car is not an Engine — it HAS an Engine; inheritance models IS-A
[ ] C. Because Engine should be final
[ ] D. Because it's slower

Why: Inheritance encodes an IS-A relationship. Car has an Engine, so composition is the right model — and it lets you swap engine implementations.

Q3. Which is NOT one of the classic four pillars?

[ ] A. Encapsulation
[ ] B. Polymorphism
[x] C. Compilation
[ ] D. Abstraction

Why: The four are Encapsulation, Inheritance, Polymorphism, Abstraction. 'Compilation' isn't a pillar.

2. Constructors, this & super Core 30 min

Constructor chaining, `this()` vs `super()`, and "why did the compiler complain there's no default constructor" — all standard fresher territory.

Key points

- A constructor has **no return type** (not even void) and its name **must** match the class name.
- If you write **any** constructor, the compiler stops generating the default no-arg one — add it back explicitly if a framework needs it (JPA/Hibernate/Jackson do).
- `this()` calls another constructor **in the same class**; `super()` calls the parent's. Both must be the **first statement** — so you can't use both in one constructor.
- If you don't write `super()` or `this()`, the compiler inserts `super()` — a call to the parent's **no-arg** constructor. If the parent has none, you get a compile error.
- Constructors are **not inherited** and **cannot be overridden** (they're not methods). They also aren't `final`/`static`/`abstract`.
- **Constructor chaining** runs Object → parent → child, so parent state is always initialised first.

Code

Chaining with this() and super()

```
class Person {
   String name; int age;
   Person() { this("Unknown", 0); }                // -> delegates to the 2-arg ctor
   Person(String name) { this(name, 0); }          // -> delegates again
   Person(String name, int age) { this.name = name; this.age = age; }
}

class Employee extends Person {
   double salary;
   Employee(String name, int age, double salary) {
       super(name, age);      // MUST be first; initialises parent state
       this.salary = salary;
   }
   Employee(double salary) {
       this("Temp", 18, salary);   // delegate to the 3-arg ctor
   }
}
```

The 'no default constructor' error everyone hits once

```
class Parent {
   Parent(String x) { }      // only a 1-arg ctor -> NO implicit no-arg ctor
}
class Child extends Parent {
   Child() {
       // Implicit super(); inserted by the compiler -> COMPILE ERROR,
       // because Parent has no no-argument constructor.
   }
}
// Fix 1: call the parent's constructor explicitly
class Child extends Parent {
   Child() { super("default"); }
}
// Fix 2: add a no-arg constructor to Parent
class Parent {
   Parent() { }
   Parent(String x) { }
}
```

this vs super — quick reference

```
class Base { int v = 1; void show() { System.out.println("Base"); } }
class Derived extends Base {
   int v = 2;
   void show() { System.out.println("Derived"); }

   void demo() {
       System.out.println(v);          // 2   -> this.v (field shadowing)
       System.out.println(this.v);     // 2
       System.out.println(super.v);    // 1   -> parent's field
       show();                         // Derived
       this.show();                    // Derived
       super.show();                   // Base  <- escape the override
   }
   Derived() { super(); }              // parent ctor, must be line 1
   Derived(int x) { this(); }          // own ctor, must be line 1
}
```

Output:

```
2
2
1
Derived
Derived
Base
```

Fields are NOT polymorphic — field access is resolved by the reference type. Only instance methods are dynamically dispatched. That's a classic trap.

Copy constructor & private constructor

```
class Point {
   int x, y;
   Point(int x, int y) { this.x = x; this.y = y; }
   Point(Point other) { this(other.x, other.y); }   // copy constructor
}

class Utility {
   private Utility() { throw new AssertionError(); }  // prevent instantiation
}
// A private ctor also enables the Singleton and Factory patterns:
class Config {
   private static final Config INSTANCE = new Config();
   private Config() { }
   public static Config getInstance() { return INSTANCE; }
}
```

SAY IT LIKE THIS IN THE INTERVIEW

"Constructors have no return type and aren't inherited or overridable. this() and super() must be the first statement, so a constructor can only do one or the other. If I don't write one, the compiler inserts a call to super() — which is why a subclass fails to compile when the parent only has a parameterised constructor."

Flashcards

Q. Why can't this() and super() appear in the same constructor?
A. Both are required to be the very first statement, so they're mutually exclusive.

Q. Does the compiler always add a default constructor?
A. Only if you declare NO constructors at all. Write any constructor and the no-arg one disappears until you add it back.

Q. Are constructors inherited or overridable?
A. Neither. Constructors aren't members in that sense — a subclass must define its own and chain to a parent one with super().

Q. What is constructor chaining?
A. Each constructor delegates up the hierarchy (super) or across (this), guaranteeing Object → parent → child initialisation order.

Q. Are fields polymorphic like methods?
A. No. Field access is resolved at compile time by the *reference* type; only instance method calls use dynamic dispatch.

Q. How do you prevent a class from being instantiated?
A. Make the constructor private (optionally throwing AssertionError) — used for utility classes and singletons.

Q. Why do JPA entities and Jackson DTOs need a no-arg constructor?
A. Frameworks instantiate the object reflectively before populating fields, so they need a no-argument constructor (it can be protected/private for JPA).

Quiz

Q1. What happens if a class has only a parameterised constructor and its subclass constructor doesn't call super(...)?

[ ] A. Compiles fine
[x] B. Compile error — the implicit super() can't find a no-arg parent constructor
[ ] C. Runtime error
[ ] D. The parent constructor is skipped

> **Why:** The compiler inserts `super();` as the first statement of every constructor that doesn't start with this()/super(...). With no no-arg parent ctor, that's a compile-time error.

Q2. What is the output? `class A { A(){ System.out.print("A"); } }` `class B extends A { B(){ System.out.print("B"); } }` `new B();`

[x] A. AB
[ ] B. BA
[ ] C. B
[ ] D. A

> **Why:** The implicit super() runs A's constructor first, then B's body — so "AB". Parent state is always built before child state.

Q3. Can a constructor be private, and why would you want that?

[ ] A. No, never
[x] B. Yes — for singletons, factories and utility classes to control instantiation
[ ] C. Yes, but only in abstract classes
[ ] D. Yes, but subclasses can't exist

> **Why:** A private constructor prevents outside instantiation (singleton, static factory) and also prevents subclassing, since no subclass ctor could call super().

Q4. What is a copy constructor?

[ ] A. A constructor that clones via Object.clone()
[x] B. A constructor taking an object of the same class and copying its state
[ ] C. The compiler-generated constructor
[ ] D. A constructor annotated @Override

> **Why:** `Point(Point other) { this(other.x, other.y); }` — an explicit, controlled alternative to clone().

3. Overloading vs Overriding & Dynamic Dispatch Must-know 40 min

The most-asked OOP question after the four pillars. Learn the rules as a table, because interviewers check them one by one. The follow-up is always: "can you override a static/private/final method?"

Key points

- **Overloading** = same method name, different parameter list, **same or different class**. Resolved at **compile time** by the declared argument types.
- **Overriding** = same signature in a subclass. Resolved at **runtime** by the actual object type (dynamic method dispatch).
- Overriding rules: same name, same parameter list, return type must be the same or a **covariant** subtype; access **cannot be narrowed** (protected → public OK, public → protected not); checked exceptions **cannot be broadened**.
- You **cannot** override `final`, `static` or `private` methods. `private` methods aren't even visible to the subclass — it's a *new* method.
- Constructors can be overloaded but not overridden. Return type alone is NOT enough to overload.
- **A subclass method can't throw a broader checked exception** than the parent's; it may throw narrower ones or none. Unchecked exceptions are unrestricted.

Reference

Aspect	Overloading	Overriding
Where	Same class (or subclass)	Subclass of an inherited method
Signature	Must DIFFER (params)	Must be IDENTICAL
Return type	Can be anything	Same or covariant subtype
Binding	Compile time (static)	Runtime (dynamic dispatch)
Access modifier	Any	Cannot be more restrictive
Exceptions	Any	Cannot throw broader checked exceptions
private/static/final	Can be overloaded	Cannot be overridden
Polymorphism	Compile-time polymorphism	Runtime polymorphism

Code

Overriding rules that compile vs fail

```
class Parent {
   public Number getValue() { return 1; }
   protected void process() throws IOException { }
   public final void locked() { }
   public static void util() { }
   private void hidden() { }
}

class Child extends Parent {
   @Override public Integer getValue() { return 2; }   // OK: covariant return
   @Override public void process() { }                 // OK: no exception, wider access
   // @Override protected void process() { }           // ERROR: narrowing access
   // @Override public void process() throws Exception // ERROR: broader checked ex
   // @Override public void locked() { }               // ERROR: final
   // @Override public static void util() { } // ERROR: static -> hiding, not override

   // NOT an override - just a new method in Child (private methods aren't inherited)
   private void hidden() { }

   // Overloading inside the same class works on return-type-independent params:
   void log(String s) { }
   void log(String s, int n) { }
   // int log(String s) { }   // ERROR: same params -> not a valid overload
}

// Dynamic dispatch in action
Parent p = new Child();
System.out.println(p.getValue());   // 2  -> Child's version runs
```

Output:
```
2
```

The classic overload-resolution trap

```
class Overloads {
   static void f(Object o) { System.out.println("Object"); }
   static void f(String s) { System.out.println("String"); }
   static void f(Integer i) { System.out.println("Integer"); }
}

Object o = "hello";
Overloads.f(o);        // "Object"  <- chosen by the DECLARED type, not the runtime type
Overloads.f("hello");  // "String"
Overloads.f(null);     // "String"  <- the MOST SPECIFIC applicable type wins

// Nothing to do with overriding: overloading is a compile-time decision.
// Overriding would have picked the runtime method; overloading cannot.
```

Output:
```
Object
String
String
```

> With `f(null)`, if there were two unrelated specific types (String and Integer) it would be ambiguous and fail to compile.

@Override — why you should never skip it

```
class Child extends Parent {
   @Override public String toString() { return "Child"; }
   // If Parent had no toString(Object) and you wrote:
   // @Override public String tostring(String s) { }  -> COMPILE ERROR (typo caught!)
}
// Without @Override the typo version would silently become a NEW method
// that is never called -> the bug nobody finds in code review.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Overloading is compile-time polymorphism — the compiler picks the method from the declared argument types. Overriding is runtime polymorphism — the JVM picks it from the actual object type. When overriding, the signature is identical, the return type can be a covariant subtype, I can widen access but never narrow it, and I can't throw broader checked exceptions. And I always add @Override so the compiler catches my typos."

Flashcards

Q. Overloading vs overriding in one sentence each?
A. Overloading: same name, different parameters, resolved at compile time. Overriding: same signature in a subclass, resolved at runtime.

Q. Can you override a private method?
A. No. Private methods aren't inherited, so a same-signature method in the subclass is a brand-new method, not an override.

Q. Can you override a static method?
A. No — a static method with the same signature hides the parent's. The call is resolved by the declared type at compile time, not dynamically.

Q. Can you change the return type when overriding?
A. Yes, if it's a covariant (subtype) return — parent returns Number, child may return Integer. Primitives must match exactly.

Q. Can an overriding method throw a broader checked exception?
A. No. It may throw the same, a narrower checked exception, none, or any unchecked exception.

Q. Can an overriding method reduce visibility?
A. No. Public must stay public; protected can become public; you can never go more restrictive.

Q. Which is chosen for `Object o = "hi"; f(o);` with f(Object) and f(String)?
A. f(Object) — overloading is resolved from the declared (static) type of the argument, not the runtime type.

Q. What does @Override actually do?
A. It's a compile-time marker annotation: the compiler errors if the method doesn't actually override anything — catching typos and signature drift.

Quiz

Q1. Which method CANNOT be overridden?

[ ] A. A public instance method
[ ] B. A protected instance method
[x] C. A final method
[ ] D. A method in a non-final class

> **Why:** final methods cannot be overridden (by design — it locks the implementation). static and private methods also can't be overridden.

Q2. Is this valid overriding? Parent: `public void m() throws IOException {}` Child: `public void m() throws Exception {}`

[ ] A. Yes, subclasses can throw anything
[x] B. No — Exception is broader than IOException (a checked exception)
[ ] C. Yes, because Exception is unchecked
[ ] D. Only if the method is abstract

> **Why:** An overriding method may not declare broader checked exceptions. It could throw FileNotFoundException (narrower) or nothing at all.

Q3. What's the result of `Parent p = new Child(); p.m();` where Child overrides m()?

[ ] A. Parent's m()
[x] B. Child's m()
[ ] C. Compile error
[ ] D. Depends on the access modifier

Why: Dynamic method dispatch: the JVM uses the runtime object type (Child) for instance method calls, regardless of the reference type.

Q4. Can two methods differ only by return type?

[ ] A. Yes, that's valid overloading
[x] B. No — the parameter list must differ
[ ] C. Yes, if one returns a subtype
[ ] D. Yes, in Java 17+

Why: Overloading requires a different parameter list (type, count or order). Return type alone doesn't distinguish methods, so it's a compile error.

Q5. What is 'method hiding'?

[x] A. Declaring a static method in a subclass with the same signature as a parent's static method
[ ] B. Using private constructors
[ ] C. Overriding with @Override
[ ] D. Shadowing a local variable

Why: Static methods are hidden, not overridden: which one runs depends on the reference type at compile time.

4. Abstract Class vs Interface Must-know 40 min

Guaranteed question, and the interesting part is how much the answer changed after Java 8. If you only memorise "interfaces are 100% abstract" you'll sound like a 2013 textbook.

Key points

- **Abstract class** — can have instance state, constructors, concrete and abstract methods. Use it for an IS-A relationship with shared code.
- **Interface** — a contract. Since Java 8 it can have `default` and `static` methods with bodies; since Java 9, `private` methods too.
- A class `extends` **one** abstract class but `implements` **many** interfaces — that's how Java gets multiple inheritance of type.
- Interface fields are implicitly `public static final`; methods are implicitly `public abstract` (unless default/static/private).
- **Functional interface** = exactly one abstract method → can be a lambda target. Mark it `@FunctionalInterface`.
- **Marker interface** = no methods at all (Serializable, Cloneable, Remote) — used to tag classes for the JVM/frameworks. Mostly replaced by annotations now.

Reference

Feature	Abstract class	Interface
Keyword	abstract class / extends	interface / implements
Multiple inheritance	No (one class)	Yes (many interfaces)
Methods	abstract + concrete	abstract, default, static, private (Java 9+)
Fields	Any (instance state)	public static final constants only
Constructors	Yes	No
Access modifiers	Any	Members are implicitly public
State	Can hold object state	No instance state
When to use	Shared code + IS-A + template method	Capability/contract, decoupled design, DI

Code

Interface superpowers since Java 8

```
interface Vehicle {
   int MAX_SPEED = 120;              // implicitly public static final

   void start();                     // implicitly public abstract

   default void honk() {             // Java 8: implementation provided
       System.out.println("Beep!");
   }

   static boolean isFast(int speed) {// Java 8: static, called as Vehicle.isFast()
       return speed > MAX_SPEED;
   }

   private void log(String msg) {    // Java 9: helper for default methods
       System.out.println("[log] " + msg);
   }

   default void service() {          // default methods can use private helpers
       log("servicing");
       System.out.println("Service done");
   }
}

class Bike implements Vehicle {
   @Override public void start() { System.out.println("Bike started"); }
   @Override public void honk() { System.out.println("Ting ting!"); }  // may override
}

Bike b = new Bike();
b.start();                // Bike started
b.honk();                 // Ting ting!
b.service();              // [log] servicing / Service done
Vehicle.isFast(150);      // true
```

Output:

```
Bike started
Ting ting!
[log] servicing
Service done
```

Default method conflict — the diamond problem and its fix

```
interface A { default void hello() { System.out.println("A"); } }
interface B { default void hello() { System.out.println("B"); } }

// class C implements A, B { }   // ERROR: inherits unrelated defaults for hello()

class C implements A, B {
   @Override public void hello() {
       A.super.hello();        // MUST disambiguate explicitly
       System.out.println("C");
   }
}
new C().hello();   // A / C

// Rule: a CLASS method always wins over an interface default.
//       Otherwise the most specific interface wins; ties must be resolved manually.
```

Output:

```
A
C
```

Template method pattern with an abstract class

```
abstract class DataProcessor {          // skeleton: fixed algorithm, variable steps
   public final void process() {        // final -> subclasses can't reorder it
       read();
       transform();                     // abstract -> subclass supplies it
       save();
   }
   private void read()   { System.out.println("reading"); }
   private void save()   { System.out.println("saving"); }
   protected abstract void transform();
}

class UpperCaseProcessor extends DataProcessor {
   @Override protected void transform() { System.out.println("UPPERCASING"); }
}
new UpperCaseProcessor().process();   // reading / UPPERCASING / saving
```

Output:
```
reading
UPPERCASING
saving
```

Functional interfaces & the built-in ones you must know

```
@FunctionalInterface      // compiler enforces: exactly ONE abstract method
interface Calculator {
   int compute(int a, int b);
   // default and static methods don't count toward the single abstract method
   default int twice(int a) { return compute(a, a); }
}

Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;
System.out.println(add.compute(3, 4));   // 7

// java.util.function - memorise these four:
Predicate<String> isEmpty  = s -> s.isEmpty();      // T -> boolean
Function<String, Integer> len = String::length;     // T -> R
Consumer<String> printer  = System.out::println;    // T -> void
Supplier<Double> random   = Math::random;           // () -> T
// Plus: BiFunction<T,U,R>, UnaryOperator<T> (T->T), BinaryOperator<T> (T,T->T)
```

Output:
```
7
```

SAY IT LIKE THIS IN THE INTERVIEW

"An abstract class models an IS-A relationship with shared state and a constructor — I use it for template-method style skeletons. An interface is a capability contract; since Java 8 it can carry default and static implementations, and a class can implement many of them, which is how Java supports multiple inheritance of type. For DI and testability I program to interfaces."

Flashcards

Q. When would you choose an abstract class over an interface?
A. When you need shared instance state, a constructor, or a common skeleton of concrete methods for closely related classes (template method pattern).

Q. Can an interface have a constructor?
A. No — interfaces have no instance state and can't be instantiated; only classes have constructors.

Q. What is a default method and why was it introduced?
A. A method with a body in an interface (Java 8). It lets interfaces evolve (e.g. Stream added to Collection) without breaking every existing implementation.

Q. How do you resolve a default-method conflict from two interfaces?
A. Override the method in the class and call the one you want explicitly: `A.super.method()`.

Q. What is a functional interface?
A. An interface with exactly one abstract method — the target type for lambdas. Annotate with @FunctionalInterface so the compiler enforces it.

Q. Name the four core java.util.function interfaces.
A. Predicate<T> (T→boolean), Function<T,R> (T→R), Consumer<T> (T→void), Supplier<T> (()→T).

Q. What is a marker interface? Give two examples.
A. An interface with no methods used to tag a class: Serializable, Cloneable, Remote. Annotations have largely replaced them.

Q. Are interface methods public by default? Fields?
A. Methods are implicitly public (abstract/default/private variants aside). Fields are implicitly public static final — constants only.

Q. Can an interface extend multiple interfaces?
A. Yes — unlike classes, an interface can extend several interfaces.

Quiz

Q1. What is the default (implicit) modifier of a field declared in an interface?

[ ] A. private
[ ] B. protected
[x] C. public static final
[ ] D. package-private

> **Why:** Interface fields are implicitly public static final — they're constants, and you must initialise them at declaration.

Q2. A class implements two interfaces that both declare `default void hello()`. What happens?

[ ] A. It compiles and picks the first interface
[x] B. Compile error unless the class overrides hello()
[ ] C. Runtime error
[ ] D. The method is randomly chosen

> **Why:** The diamond conflict must be resolved explicitly: override hello() and delegate with `A.super.hello()` or `B.super.hello()`.

Q3. Which is true about a class that extends a base class and implements an interface, where both provide the same method?

[ ] A. The interface default wins
[x] B. The class (superclass) method wins — classes beat interfaces
[ ] C. Compile error
[ ] D. The most specific interface wins

> **Why:** The rule is: class wins over interface. A superclass method takes precedence over any inherited default method.

Q4. Can an interface contain a private method?

[ ] A. No, never
[x] B. Yes, since Java 9, as a helper for default methods
[ ] C. Yes, since Java 8
[ ] D. Only static private methods

> **Why:** Java 9 added private interface methods so default methods can share helper code without exposing it.

Q5. Which of these is a valid functional interface?

[ ] A. interface X { void a(); void b(); }
[x] B. interface X { void a(); default void b(){} static void c(){} }
[ ] C. interface X { }
[ ] D. interface X { void a(); void a(int i); }

Why: Exactly one abstract method. default and static methods don't count, so option 2 qualifies. Option 4 has two abstract methods.

5. Object Methods & the equals/hashCode Contract Must-know 40 min

`equals` and `hashCode` are the most technically-demanding fresher question there is. Learn the contract as five numbered rules and you'll never be caught out — and it explains why HashMap misbehaves when you get it wrong.

Key points

- Every class inherits from `Object`: `toString()`, `equals()`, `hashCode()`, `getClass()`, `clone()`, `notify()/wait()`, `finalize()` (deprecated since 9).
- **The contract**: if `a.equals(b)` then `a.hashCode() == b.hashCode()` — always. The reverse is NOT required (hash collisions are allowed).
- If you override `equals()`, you MUST override `hashCode()`, or HashMap/HashSet will lose your objects.
- `equals()` must be **reflexive, symmetric, transitive, consistent**, and `x.equals(null)` must be `false`.
- Use `Objects.equals(a, b)` and `Objects.hash(...)` — null-safe one-liners.
- **Never use a mutable field in hashCode/equals** — if the field changes after insertion into a HashSet, the object becomes unreachable.

Reference

Object method	Default behaviour	Why/when you override
toString()	ClassName@hexHash	Logging and debugging — always override for readability
equals(Object)	Reference equality (==)	When logical equality matters (value objects, DTOs)
hashCode()	Identity-based int	MUST override whenever equals is overridden
clone()	Shallow field copy	Rarely — prefer copy constructors; requires Cloneable
getClass()	Returns the runtime Class	final — cannot be overridden
finalize()	Called by GC before reclaim	Deprecated (Java 9) — use try-with-resources / Cleaner

Code

A correct equals + hashCode (Java 7+ style)

```
class Employee {
   private int id;
   private String name;

   @Override
   public boolean equals(Object o) {
       if (this == o) return true;              // 1. identity shortcut
       if (!(o instanceof Employee)) return false; // 2. type check (also handles null)
       Employee that = (Employee) o;
       return id == that.id && Objects.equals(name, that.name);
   }

   @Override
   public int hashCode() {
       return Objects.hash(id, name);           // same fields as equals!
   }

   @Override
   public String toString() {
       return "Employee{id=" + id + ", name='" + name + "'}";
   }
}
// Use `instanceof` (not getClass()) so subclasses can still be equal.
// Use `==` for primitives, Objects.equals() for references.
```

What breaks when you forget hashCode() — the HashSet bug

```
class Bad {
   int id;
   Bad(int id) { this.id = id; }
   @Override public boolean equals(Object o) {
       return o instanceof Bad && ((Bad) o).id == this.id;
   }
   // hashCode() NOT overridden -> uses Object's identity hash
}

Set<Bad> set = new HashSet<>();
set.add(new Bad(1));
System.out.println(set.contains(new Bad(1)));   // false!!!
// Two equal objects land in different buckets because their identity
// hashCodes differ, so HashSet never finds the match.

// With hashCode() implemented correctly -> true
```

Output:
```
false
```

> This is the canonical interview demo. Say it out loud: "equal objects must have equal hash codes, otherwise hash-based collections break."

Mutable-field trap: losing an object inside a HashSet

```
class Mutable {
   int id;
   Mutable(int id) { this.id = id; }
   @Override public int hashCode() { return Objects.hash(id); }
   @Override public boolean equals(Object o) {
       return o instanceof Mutable && ((Mutable) o).id == this.id;
   }
}

Set<Mutable> set = new HashSet<>();
Mutable m = new Mutable(1);
set.add(m);
m.id = 99;                          // mutating a field used in hashCode
System.out.println(set.contains(m));         // false - different bucket now
set.remove(m);                               // fails -> the entry is orphaned
// Lesson: use FINAL fields in equals/hashCode, or never mutate keys in a set.
```

Output:
```
false
```

Shallow vs deep copy (clone and copy constructors)

```
class Address { String city; Address(String c) { city = c; } }

class Person implements Cloneable {
   String name;
   Address addr;                // reference field -> shallow copy shares it!

   @Override protected Object clone() throws CloneNotSupportedException {
       return super.clone();     // shallow: addr is shared
   }
   // Deep copy version:
   Person deepCopy() {
       Person p = new Person();
       p.name = this.name;
       p.addr = new Address(this.addr.city);   // clone the nested object
       return p;
   }
}

Person p1 = new Person(); p1.addr = new Address("Hyd");
Person p2 = (Person) p1.clone();
p2.addr.city = "Blr";
System.out.println(p1.addr.city);   // Blr  <- shallow copy leaked the change!
```

Output:
```
Blr
```

> Shallow copy duplicates the top-level object but shares nested references. Deep copy recursively duplicates everything. Prefer copy constructors — clone()/Cloneable is widely considered broken.

SAY IT LIKE THIS IN THE INTERVIEW

"Every class inherits from Object. The key contract: if two objects are equal their hash codes must be equal, so I always override equals and hashCode together, using the same fields. I use Objects.equals and Objects.hash for null safety, instanceof for the type check, and I keep those fields immutable so an object can't go missing inside a HashMap."

Flashcards

Q. State the equals/hashCode contract.
A. If a.equals(b) then a.hashCode() == b.hashCode(). The converse isn't required (collisions are legal). Equal hashCodes don't imply equality.

Q. What breaks if you override equals but not hashCode?
A. Hash-based collections (HashMap, HashSet) put equal objects in different buckets, so contains/get/remove return false even though the object is present.

Q. List the 5 properties equals() must satisfy.
A. Reflexive (x.equals(x)), symmetric (x.equals(y) ⇒ y.equals(x)), transitive, consistent across calls, and x.equals(null) is false.

Q. Why use instanceof instead of getClass() in equals()?
A. getClass() breaks equality for subclasses and proxies (Hibernate/Spring use proxies); instanceof allows subtype equality. Either is acceptable if you're consistent, but instanceof is the common convention.

Q. Shallow copy vs deep copy?
A. Shallow duplicates the top-level object but shares nested references; deep recursively copies all referenced objects.

Q. Why is clone() considered broken?
A. Cloneable is a marker interface with no clone() method, clone() is protected, it bypasses constructors, and it does a shallow copy by default. Prefer copy constructors or factory methods.

Q. Is finalize() recommended?
A. No — deprecated since Java 9. It's unpredictable and slows GC. Use try-with-resources and AutoCloseable.

Q. Which Object methods are final?
A. getClass(), notify(), notifyAll(), wait() — they can't be overridden.

Quiz

Q1. You override equals() but not hashCode(). What is the result?

[ ] A. Compile error
[x] B. Hash-based collections misbehave — equal objects may not be found
[ ] C. Nothing; hashCode is optional
[ ] D. equals() stops working

> **Why:** It compiles, but you've broken the contract: two equal objects get different (identity) hash codes, so HashSet/HashMap lookups fail.

Q2. Which equals() property is violated here? `class A { int x; public boolean equals(Object o){ return o instanceof A && ((A)o).x == x; } }` `class B extends A { int y; public boolean equals(Object o){ return o instanceof B && ((B)o).y == y; } }`

[ ] A. Reflexive
[x] B. Symmetric
[ ] C. Transitive
[ ] D. Consistent

> **Why:** `a.equals(b)` can be true (b is an A, x matches) while `b.equals(a)` is false (a is not a B). Also a transitivity problem. Fix by using getClass() or `canEqual` — which is why many teams let the IDE generate equals.

Q3. What does `Object.hashCode()` return by default?

[ ] A. 0
[x] B. An identity-based hash (not necessarily the memory address)
[ ] C. The class name hash
[ ] D. A random number each call

> **Why:** The default is derived from the object's identity — typically the address converted to an int, though the spec allows other schemes. It stays stable for the object's lifetime.

Q4. After adding an object to a HashSet, you mutate a field used in hashCode(). What happens?

[ ] A. Nothing — the set updates the bucket
[x] B. The object becomes unfindable/removable — it's in the wrong bucket
[ ] C. ConcurrentModificationException
[ ] D. It's automatically rehashed

> **Why:** HashMap never rehashes existing keys. The entry sits in the bucket computed from the old hash, so lookups with the new hash miss it — a memory leak in practice.

Q5. What's the correct way to compare two objects that might be null?

[ ] A. a.equals(b)
[x] B. Objects.equals(a, b)
[ ] C. a == b
[ ] D. a.hashCode() == b.hashCode()

> **Why:** Objects.equals handles nulls on either side (returns true if both are null). a.equals(b) throws NPE when a is null.

6. Exception Handling Must-know 45 min

A whole round can be built on exceptions: the hierarchy, checked vs unchecked, and the `finally` / `return` gotchas. Learn the hierarchy diagram and the six gotcha outputs.

Key points

- `Throwable` → `Error` (JVM-level, don't catch) and `Exception` → `RuntimeException` (unchecked).
- **Checked** exceptions (IOException, SQLException, ClassNotFoundException) are verified at **compile time** — you must catch or declare `throws`.
- **Unchecked** (RuntimeException and its children: NPE, ArrayIndexOutOfBounds, ArithmeticException, IllegalArgumentException, ClassCastException, NumberFormatException, ConcurrentModificationException) are not checked by the compiler.
- `Error` (OutOfMemoryError, StackOverflowError, NoClassDefFoundError) is fatal — never catch it.
- `finally` runs even after a `return` — but a `return` inside `finally` **overrides** the earlier return.
- **try-with-resources** (Java 7) auto-closes anything `AutoCloseable`; resources are closed in reverse order, and suppressed exceptions are attached to the primary one.

Reference

Type	Checked at compile time?	Examples	Handle or declare?
Checked Exception	Yes	IOException, SQLException, FileNotFoundException, InterruptedException, ClassNotFoundException	Must catch or declare throws
Unchecked (RuntimeException)	No	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException, ClassCastException, IllegalArgumentException, NumberFormatException	Optional — usually fix the bug instead
Error	No	OutOfMemoryError, StackOverflowError, NoClassDefFoundError	Do not catch

Code

The hierarchy, as text

```
Throwable
├── Error                    (unchecked, fatal - never catch)
│   ├── OutOfMemoryError
│   ├── StackOverflowError
│   └── NoClassDefFoundError
└── Exception
    ├── RuntimeException      (unchecked)
    │   ├── NullPointerException
    │   ├── ArithmeticException          <- / by zero (int)
    │   ├── ArrayIndexOutOfBoundsException
    │   ├── ClassCastException
    │   ├── IllegalArgumentException
    │   │   └── NumberFormatException    <- Integer.parseInt("abc")
    │   ├── IllegalStateException
    │   └── ConcurrentModificationException
    └── checked exceptions
        ├── IOException
        │   └── FileNotFoundException
        ├── SQLException
        ├── ClassNotFoundException
        ├── InterruptedException
        └── ParseException
```

The six outputs you must know by heart

```
// 1. finally overrides a return
static int f1() {
   try { return 1; }
   finally { return 2; }        // WARNING: never do this
}
System.out.println(f1());        // 2

// 2. finally runs even when an exception escapes
static void f2() {
   try { throw new RuntimeException("boom"); }
   finally { System.out.println("finally ran"); }
}
// prints "finally ran", then the exception propagates

// 3. return value is SNAPSHOTTED before finally mutates it
static int f3() {
   int x = 10;
   try { return x; }            // the value 10 is remembered here
   finally { x = 20; }          // too late
}
System.out.println(f3());        // 10

// 4. System.exit() is the one thing finally can't stop
try { System.exit(0); } finally { System.out.println("never printed"); }

// 5. catch order: subclass BEFORE superclass
try { ... }
catch (FileNotFoundException e) { }   // specific first
catch (IOException e) { }             // general after
// catch (IOException e){} catch (FileNotFoundException e){}  -> COMPILE ERROR

// 6. multi-catch (Java 7) - the parameter is implicitly final
catch (IOException | SQLException e) { log(e); }
```

```
Output:
2
10
```

try-with-resources — the modern way, and why it's better

```
// OLD: verbose, and a bug if close() itself throws
BufferedReader br = null;
try { br = new BufferedReader(new FileReader("a.txt")); }
finally { if (br != null) try { br.close(); } catch (IOException e) { } }

// NEW (Java 7+): anything AutoCloseable closes automatically, in reverse order
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"));
    Connection conn = DriverManager.getConnection(url)) {
   return br.readLine();
}   // conn closes first, then br - even if the body threw

// Java 9+: effectively-final resources don't need redeclaring
static void copy(BufferedReader in, PrintWriter out) throws IOException {
   try (in; out) { out.println(in.readLine()); }
}

// Write your own AutoCloseable
class MyResource implements AutoCloseable {
   public void close() { System.out.println("closed"); }
}
```

Suppressed exceptions: if both the body and close() throw, the close() exception is attached via addSuppressed() to the body's exception, so you don't lose it.

throw vs throws, and custom exceptions

```
// throws = DECLARATION (part of the signature, checked exceptions only)
public void readFile(String path) throws IOException {
   if (path == null) throw new IllegalArgumentException("path required"); // unchecked
   Files.readAllLines(Path.of(path));
}

// Custom CHECKED exception -> extends Exception
class InsufficientFundsException extends Exception {
   public InsufficientFundsException(String msg) { super(msg); }
}
// Custom UNCHECKED exception -> extends RuntimeException
class InvalidOrderException extends RuntimeException {
   private final String orderId;                    // carry context
   public InvalidOrderException(String orderId, String msg) {
       super(msg);
       this.orderId = orderId;
   }
}

// Exception chaining: always preserve the cause
try { repo.save(order); }
catch (SQLException e) {
   throw new OrderSaveException("Could not save order " + order.getId(), e);
   //                                                                  ^ cause
}
```

Five exception-handling anti-patterns they'll ask you to spot

```
// 1. Swallowing the exception
try { doWork(); } catch (Exception e) { /* nothing */ }        // worst

// 2. Catching Throwable / Error
try { doWork(); } catch (Throwable t) { }                      // hides OOM

// 3. Logging AND rethrowing (double reporting)
catch (IOException e) { log.error("fail", e); throw e; }       // log at the boundary only

// 4. Throwing away the cause
catch (SQLException e) { throw new ServiceException("failed"); }  // lost cause!

// 5. Using exceptions for normal control flow
try { int v = map.get(key); } catch (NullPointerException e) { }  // use containsKey()

// Preferred: catch specific, log at the boundary, preserve the cause,
// and let unchecked exceptions from bugs propagate.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Throwable splits into Error, which I never catch, and Exception. Exceptions are checked — the compiler forces me to catch or declare them — or unchecked RuntimeExceptions, which usually indicate a bug I should fix rather than catch. I use try-with-resources so resources always close, I catch specific types from most- to least-specific, and when I wrap an exception I always pass the original as the cause."

Flashcards

Q. Difference between checked and unchecked exceptions?
A. Checked (IOException, SQLException) are verified at compile time — catch or declare. Unchecked (RuntimeException and subclasses) aren't; they usually signal a programming error.

Q. What is the difference between Error and Exception?
A. Error is a serious JVM-level failure (OutOfMemoryError, StackOverflowError) you should not catch. Exception represents recoverable conditions an application can handle.

Q. Does finally always execute?
A. Yes, except when System.exit() is called, the JVM crashes, the thread is killed, or a daemon thread's finally when the JVM exits.

Q. What happens if both try and finally have a return?
A. The finally block's return wins and the try's return is discarded (and any exception in try is swallowed). Never return from finally.

Q. throw vs throws?
A. throw is the statement that actually throws an exception object; throws is the method declaration listing which checked exceptions may propagate.

Q. What is try-with-resources and what's the requirement?
A. A try that auto-closes resources declared in parentheses; they must implement AutoCloseable. Closed in reverse order, even if the body throws.

Q. What is a suppressed exception?
A. When the try body and close() both throw, the close() exception is attached to the primary one via addSuppressed() instead of replacing it.

Q. Can you catch multiple exception types in one block?
A. Yes (Java 7): `catch (IOException | SQLException e)`. The parameter is implicitly final, and the types can't be related by subclassing.

Q. Is NumberFormatException checked or unchecked?
A. Unchecked — it extends IllegalArgumentException, which extends RuntimeException.

Q. Why is catching Exception or Throwable bad practice?
A. It swallows errors you can't handle (like OutOfMemoryError) and hides bugs. Catch the most specific type you can actually recover from.

Quiz

Q1. What does this print? `static int m(){ try { return 1; } finally { return 2; } }`

[ ] A. 1
[x] B. 2
[ ] C. Compile error
[ ] D. 1 then 2

> **Why:** A return in finally overrides the try's return — and swallows any exception too. It's a well-known anti-pattern.

Q2. What is the output? `static int m(){ int x=10; try { return x; } finally { x=20; } }`

[x] A. 10
[ ] B. 20
[ ] C. 30
[ ] D. Compile error

> **Why:** The return value is evaluated and saved before finally runs, so changing x afterwards has no effect on what's returned.

Q3. Which is a checked exception?

[ ] A. NullPointerException
[x] B. IOException
[ ] C. ArithmeticException
[ ] D. IllegalArgumentException

> **Why:** IOException (and its subclasses) are checked. The other three are RuntimeExceptions, i.e. unchecked.

Q4. What is wrong with `catch (Exception e) { }` after `catch (IOException e) { }`?

[ ] A. Nothing, it's fine
[x] B. Nothing — but the reverse order is a compile error
[ ] C. Both are compile errors
[ ] D. It throws at runtime

> **Why:** Catch blocks must go from most specific to most general. `catch (IOException)` after `catch (Exception)` is unreachable → compile error.

Q5. In try-with-resources, if the body throws AND close() throws, which exception surfaces?

[ ] A. The close() exception, the body one is lost

[x] B. The body exception, with the close() exception attached as suppressed

[ ] C. Both are thrown

[ ] D. Neither — they cancel out

Why: The primary (body) exception propagates and the close() exception is added via addSuppressed(), so no information is lost.

Q6. Which statement about finally is FALSE?

[ ] A. It runs even if the try block returns

[ ] B. It runs even if an exception is thrown

[x] C. It runs even after System.exit(0)

[ ] D. It's used for cleanup like closing connections

Why: System.exit() terminates the JVM immediately; finally blocks do not run (that's why shutdown hooks exist).

7. Collections Framework: List & Set Must-know 45 min

The Collections Framework is the single most-tested area in Java interviews. Learn the interface hierarchy, then the implementation trade-offs: ArrayList vs LinkedList, HashSet vs TreeSet vs LinkedHashSet.

Key points

- `Collection` → `List`, `Set`, `Queue`. `Map` is separate (it does NOT extend Collection).
- **ArrayList** = dynamic array. Fast random access O(1), slow middle insert/delete O(n). Default capacity 10, grows ×1.5.
- **LinkedList** = doubly-linked. Fast insert/delete at the ends O(1), slow random access O(n). Also implements Deque.
- **Vector** = legacy, synchronized, grows ×2. Use `ArrayList` or `CopyOnWriteArrayList` instead.
- **HashSet** = hash table, no order, O(1). **LinkedHashSet** = insertion order preserved. **TreeSet** = sorted (red-black tree), O(log n).
- HashSet uses `hashCode()` to pick the bucket and `equals()` to detect duplicates — which is why the equals/hashCode contract matters.

Reference

Implementation	Order	get/contains	add/remove	Use when
ArrayList	Insertion	O(1) by index	O(n) in the middle, O(1) at end	Default list — random access, iteration
LinkedList	Insertion	O(n)	O(1) at ends (with a reference)	Frequent insert/delete at both ends; queue/deque
Vector	Insertion	O(1)	O(n)	Legacy — synchronized, avoid in new code
HashSet	None	O(1)	O(1)	Fast membership, unique elements, no order needed
LinkedHashSet	Insertion	O(1)	O(1)	Unique + predictable iteration order
TreeSet	Sorted (natural/Comparator)	O(log n)	O(log n)	Need sorted unique elements, range queries

Code

The hierarchy, as text you can redraw in the interview

```
Iterable
└── Collection
    ├── List          (ordered, indexed, allows duplicates)
    │   ├── ArrayList
    │   ├── LinkedList   (also implements Deque)
    │   └── Vector / Stack (legacy)
    ├── Set           (no duplicates)
    │   ├── HashSet
    │   │   └── LinkedHashSet
    │   └── SortedSet
    │       └── NavigableSet
    │           └── TreeSet
    └── Queue         (FIFO / priority)
        ├── PriorityQueue
        ├── ArrayDeque
        └── Deque (double-ended)
            ├── ArrayDeque
            └── LinkedList

Map  (SEPARATE hierarchy - does NOT extend Collection)
├── HashMap
│   └── LinkedHashMap
├── Hashtable (legacy, synchronized)
├── SortedMap -> NavigableMap -> TreeMap
└── ConcurrentHashMap
```

List: create, mutate, iterate

```
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Spring");
list.add(1, "JPA");          // insert at index -> [Java, JPA, Spring]
list.set(0, "Core Java");    // replace
list.get(0);                 // "Core Java" - O(1)
list.remove("JPA");          // by object
list.remove(0);              // by index (careful: remove(int) vs remove(Object))
list.contains("Spring");     // true (uses equals)
list.size();
list.isEmpty();
list.indexOf("Spring");
list.clear();

// Creating lists
List<String> fixed  = Arrays.asList("a","b","c");   // fixed-size, backed by array
List<String> immut  = List.of("a","b","c");         // Java 9, truly immutable, no nulls
List<String> mutable= new ArrayList<>(List.of("a","b"));
List<Integer> nums  = new ArrayList<>(Arrays.asList(3,1,2));
Collections.sort(nums);                              // [1,2,3]

// Three ways to iterate
for (int i = 0; i < list.size(); i++) { }            // need the index
for (String s : list) { }                            // enhanced for (cleanest)
list.forEach(s -> System.out.println(s));            // Java 8
Iterator<String> it = list.iterator();
while (it.hasNext()) { if (it.next().equals("x")) it.remove(); }  // safe removal
```

Set: deduplication and how HashSet decides 'duplicate'

```
Set<String> set = new HashSet<>();
set.add("a"); set.add("b"); set.add("a");    // the second "a" is ignored
System.out.println(set.size());              // 2
System.out.println(set.add("a"));            // false -> already present

// Order guarantees
new HashSet<>(List.of(3,1,2));         // iteration order UNSPECIFIED
new LinkedHashSet<>(List.of(3,1,2));   // 3,1,2 (insertion order)
new TreeSet<>(List.of(3,1,2));         // 1,2,3 (sorted)

// TreeSet with a custom comparator + range queries
NavigableSet<Integer> ts = new TreeSet<>(List.of(10, 5, 20, 15));
ts.first();  ts.last();                // 5, 20
ts.lower(15);  ts.higher(15);          // 10, 20
ts.headSet(15); ts.tailSet(15);        // [5,10], [15,20]
ts.descendingSet();                    // reverse view

// Dedupe a list quickly:
List<String> unique = new ArrayList<>(new HashSet<>(duplicates));

// Custom objects in a HashSet -> must implement equals + hashCode
Set<Employee> emps = new HashSet<>();
emps.add(new Employee(1, "A"));
emps.add(new Employee(1, "A"));   // deduped ONLY if equals/hashCode are correct
```

```
Output:
2
false
```

Common coding patterns with collections

```
List<Integer> nums = List.of(5, 3, 8, 1, 9, 3);

// Frequency map (the most useful pattern in coding rounds)
Map<Integer, Long> freq = nums.stream()
       .collect(Collectors.groupingBy(n -> n, Collectors.counting()));
// {1=1, 3=2, 5=1, 8=1, 9=1}

// Remove duplicates while keeping order
List<Integer> deduped = nums.stream().distinct().toList();

// Find the first duplicate
Set<Integer> seen = new HashSet<>();
Integer dup = nums.stream().filter(n -> !seen.add(n)).findFirst().orElse(null); // 3

// Min / max / reverse
Collections.max(nums); Collections.min(nums); Collections.reverse(list);
Collections.frequency(nums, 3);   // 2

// Sorting by a field
employees.sort(Comparator.comparing(Employee::getName)
                         .thenComparing(Employee::getSalary, Comparator.reverseOrder()));

// Null-safe, defensive copies
List<String> safe = list == null ? List.of() : List.copyOf(list);
```

SAY IT LIKE THIS IN THE INTERVIEW

"Collection splits into List, Set and Queue, and Map sits in its own hierarchy. ArrayList is my default — backed by a resizable array with O(1) random access; LinkedList only wins for frequent insertions at both ends. For unique elements I use HashSet for speed, LinkedHashSet when I need insertion order, and TreeSet when I need them sorted. HashSet relies on hashCode to find the bucket and equals to confirm the duplicate, which is why I override them together."

Flashcards

Q. ArrayList vs LinkedList — when do you use each?
A. ArrayList (dynamic array): O(1) random access, O(n) insert/delete in the middle — the default. LinkedList (doubly-linked): O(1) insert/delete at the ends, O(n) access — good as a deque/queue.

Q. Does Map extend Collection?
A. No. Map has its own hierarchy and is not a Collection. (You bridge them with keySet(), values(), entrySet().)

Q. HashSet vs LinkedHashSet vs TreeSet?
A. HashSet: fastest, no order. LinkedHashSet: insertion order. TreeSet: sorted order, O(log n), supports range queries.

Q. How does HashSet detect a duplicate?
A. It uses hashCode() to pick the bucket, then equals() to check whether an equal element already exists in that bucket.

Q. What is the default capacity and growth factor of ArrayList?
A. Default capacity 10; when full it grows by ~1.5x (oldCapacity + oldCapacity >> 1) and copies the array.

Q. Vector vs ArrayList?
A. Vector is legacy and synchronizes every method (slower, grows 2x). ArrayList is not synchronized — use Collections.synchronizedList or CopyOnWriteArrayList if you need thread safety.

Q. How do you make a list immutable?
A. List.of(...) (Java 9) or Collections.unmodifiableList(...) / List.copyOf(...) for a defensive snapshot.

Q. Why does remove() behave oddly on a List<Integer>?
A. Overload ambiguity: remove(int) removes by index, remove(Object) removes by value. `list.remove(1)` removes index 1; you need `list.remove(Integer.valueOf(1))` to remove the value 1.

Q. What's the difference between Collection and Collections?
A. Collection is the root interface of the framework; Collections is a utility class with static helpers (sort, max, reverse, synchronizedList, unmodifiableList).

Quiz

Q1. You need fast lookup by index and mostly read. Which do you choose?

[ ] A. LinkedList
[x] B. ArrayList
[ ] C. TreeSet
[ ] D. HashSet

> **Why:** ArrayList gives O(1) random access by index and excellent cache locality for iteration — the right default for read-heavy, index-access workloads.

Q2. Which Set preserves insertion order?

[ ] A. HashSet
[ ] B. TreeSet
[x] C. LinkedHashSet
[ ] D. EnumSet

> **Why:** LinkedHashSet maintains a linked list of entries alongside the hash table, so iteration follows insertion order (at a small memory cost).

Q3. What is the time complexity of `contains()` on an ArrayList?

[ ] A. O(1)
[ ] B. O(log n)
[x] C. O(n)
[ ] D. O(n log n)

> **Why:** contains() does a linear scan calling equals() on each element → O(n). (HashSet.contains is O(1) — that's the usual reason to switch.)

Q4. What happens when you add a duplicate element to a HashSet?

[ ] A. It throws an exception
[ ] B. The old element is replaced

[x] C. add() returns false and the set is unchanged

[ ] D. It's added anyway

> **Why:** Set.add returns false if the element is already present (as determined by hashCode + equals); the set is unchanged.

Q5. What does `List.of("a", "b")` return and what happens on add()?

[ ] A. A mutable ArrayList

[x] B. An immutable list; add() throws UnsupportedOperationException

[ ] C. A fixed-size list; add() silently fails

[ ] D. A synchronized list

> **Why:** List.of (Java 9) returns a truly immutable list that also rejects nulls. add/remove/set throw UnsupportedOperationException.

8. Map & HashMap Internals Must-know 50 min

"Explain how HashMap works internally" is the most common Java collection question at any level. Get the story straight: hash → bucket → equals → treeify → resize.

Key points

- `HashMap` stores **Node<K,V>** objects in an array of **buckets**. Put: hash the key → index = `hash & (n-1)` → walk that bucket → `equals` to find an existing key.
- **Collision handling**: before Java 8, only chaining (linked list). Since Java 8, a bucket with **>8** nodes AND table size **≥64** is converted to a **red-black tree** (O(log n) instead of O(n)); it drops back to a list when it shrinks below 6.
- **Load factor 0.75**, default capacity 16. When `size > capacity * 0.75`, the table **doubles** and rehashes — that's the expensive operation to avoid by pre-sizing.
- HashMap allows **one null key** and multiple null values; `Hashtable` and `ConcurrentHashMap` allow **neither**.
- HashMap is **not thread-safe**. `Hashtable` is synchronized (legacy). `ConcurrentHashMap` is the modern choice (segment/-node-level locking, no nulls).
- **Keys must be immutable** — mutating a key after insertion makes the entry unreachable.

Reference

Map impl	Order	Null key?	Thread-safe?	Notes
HashMap	None	Yes (one)	No	Default. O(1) average get/put
LinkedHashMap	Insertion / access	Yes	No	Perfect for LRU caches (access-order + removeEldestEntry)
TreeMap	Sorted by key	No (NPE)	No	Red-black tree, O(log n), range queries
Hashtable	None	No	Yes (whole-map lock)	Legacy — use ConcurrentHashMap
ConcurrentHashMap	None	No	Yes (fine-grained)	High concurrency, no nulls, weakly consistent iterators

Code

How HashMap works, step by step (the answer to recite)

```
// 1. hash(key) -> spread the bits so bad hashCodes don't cluster
static final int hash(Object key) {
   int h;
   return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
// 2. index = hash & (n - 1)   <- fast modulo because n is a power of two
// 3. Walk the bucket:
//      - if a node has the same hash && (k == key || key.equals(k))  -> UPDATE value
//      - else append a new node (collision -> chaining)
// 4. If the bucket exceeds TREEIFY_THRESHOLD (8) and the table is >= 64
//      -> convert the chain to a red-black tree (worst case O(log n))
// 5. If ++size > threshold (capacity * 0.75) -> resize: new table 2x, rehash all

Map<String, Integer> map = new HashMap<>();   // capacity 16, load factor 0.75
map.put("a", 1);
map.put("a", 2);            // same key -> value replaced, put returns the old value (1)
map.get("a");               // 2
map.getOrDefault("zz", 0);  // 0  <- avoids NPE, very handy
map.putIfAbsent("b", 10);   // only if absent
map.computeIfAbsent("c", k -> k.length());     // lazy compute, great for groupings
map.merge("a", 5, Integer::sum);               // 7  <- counting pattern
map.get("missing");         // null (careful: also null if the value is null)
```

Iterating a Map — and the only correct way to remove during iteration

```
Map<String, Integer> map = new HashMap<>(Map.of("a",1,"b",2,"c",3));

// Best: entrySet (one lookup per entry, not two)
for (Map.Entry<String, Integer> e : map.entrySet()) {
   System.out.println(e.getKey() + " -> " + e.getValue());
}
map.forEach((k, v) -> System.out.println(k + " -> " + v));   // Java 8

// WRONG - ConcurrentModificationException
// for (String k : map.keySet()) { if (k.equals("a")) map.remove(k); }

// RIGHT - use the iterator's own remove()
Iterator<Map.Entry<String,Integer>> it = map.entrySet().iterator();
while (it.hasNext()) { if (it.next().getKey().equals("a")) it.remove(); }

// RIGHT (Java 8) - removeIf on the entry set
map.entrySet().removeIf(e -> e.getValue() < 2);
// or: map.keySet().removeIf(k -> k.startsWith("a"));
```

Pre-sizing and the LRU cache trick

```
// Pre-size to avoid rehashing: expected / 0.75 + 1
int expected = 1000;
Map<String, String> m = new HashMap<>((int) (expected / 0.75f) + 1);

// An LRU cache in 6 lines using LinkedHashMap's access-order mode
class LruCache<K, V> extends LinkedHashMap<K, V> {
   private final int max;
   LruCache(int max) {
       super((int) (max / 0.75f) + 1, 0.75f, true); // true = access order
       this.max = max;
   }
   @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
       return size() > max;      // evict the least-recently-used entry
   }
}
```

ConcurrentHashMap vs synchronizedMap vs Hashtable

```
Map<String,Integer> sync = Collections.synchronizedMap(new HashMap<>());
// Every method locks the whole map -> safe but a concurrency bottleneck.
// IMPORTANT: you must manually synchronize while iterating:
synchronized (sync) { for (String k : sync.keySet()) { } }

Map<String,Integer> chm = new ConcurrentHashMap<>();
// Locks only the affected bin (or uses CAS) -> readers never block.
// Iterators are weakly consistent: no ConcurrentModificationException.
chm.put("a", 1);
chm.compute("a", (k, v) -> v == null ? 1 : v + 1);   // atomic increment
chm.merge("hits", 1, Integer::sum);                  // atomic counter
// chm.put(null, 1);  -> NullPointerException (nulls are forbidden)

// Fail-safe iteration: CHM's iterator reflects some point in time, not live changes
```

SAY IT LIKE THIS IN THE INTERVIEW

"HashMap is an array of buckets indexed by hash(key) & (n-1). On a collision the entries are chained in a linked list, and since Java 8 a bucket with more than 8 entries is treeified into a red-black tree so the worst case stays O(log n). It resizes by doubling when size exceeds 0.75 × capacity. Keys must implement equals and hashCode properly and be immutable. For concurrency I'd use ConcurrentHashMap, not Hashtable — it locks per-bin and doesn't allow nulls."

Flashcards

Q. Explain HashMap's put() internals.
A. hash(key) → bucket index = hash & (n-1) → scan the bucket: if an equal key exists (same hash && equals) replace the value, else append a node. Treeify at >8 entries, resize (2x) past 0.75 load.

Q. How does HashMap handle collisions?
A. Separate chaining (linked list per bucket) since always; since Java 8 a bucket with >8 nodes and table ≥64 becomes a red-black tree, degrading back to a list below 6.

Q. Default capacity and load factor of HashMap?
A. Capacity 16, load factor 0.75 → resizes (doubles + rehashes) once the 13th entry is added.

Q. Why must HashMap keys be immutable?
A. The bucket is computed from the key's hash at insertion. Mutating the key changes its hash, so get/remove look in the wrong bucket and the entry is orphaned (a leak).

Q. HashMap vs Hashtable?
A. HashMap: unsynchronized, allows one null key and null values, modern. Hashtable: legacy, every method synchronized (slow), no nulls allowed.

Q. HashMap vs ConcurrentHashMap?
A. CHM is thread-safe with per-bin locking/CAS, doesn't allow nulls, and its iterators are weakly consistent (never throw ConcurrentModificationException). HashMap isn't thread-safe.

Q. How do you safely remove from a map while iterating?
A. Use iterator.remove(), or entrySet().removeIf(...) — never map.remove() inside an enhanced for loop (ConcurrentModificationException).

Q. How do you implement an LRU cache in Java?
A. Extend LinkedHashMap in access-order mode (constructor flag true) and override removeEldestEntry to evict past a max size. Or use Caffeine/Guava in production.

Q. Time complexity of HashMap get/put?
A. O(1) average, O(log n) worst case since Java 8 (treeified buckets); a badly-distributed hashCode degrades everything to one bucket.

Q. What does computeIfAbsent do?
A. If the key is absent (or maps to null), computes the value from the function, inserts it and returns it — atomic on ConcurrentHashMap. Perfect for building Map<K, List<V>> groupings.

Quiz

Q1. At how many colliding entries does Java 8+ HashMap treeify a bucket?

[ ] A. > 4

[x] B. > 8 (and table size ≥ 64)
[ ] C. > 16
[ ] D. > 32

Why: TREEIFY_THRESHOLD is 8, treeifying only kicks in when the table is at least MIN_TREEIFY_CAPACITY = 64; below 6 it de-treeifies back to a list.

Q2. Which collection allows a null key?

[ ] A. Hashtable
[ ] B. ConcurrentHashMap
[x] C. HashMap
[ ] D. TreeMap

Why: HashMap permits one null key (hashed to bucket 0) and any number of null values. Hashtable and ConcurrentHashMap throw NPE; TreeMap throws NPE with a natural-ordering comparator.

Q3. What happens if you modify a HashMap key after putting it in the map?

[ ] A. The map rehashes the entry
[x] B. The entry becomes unreachable — get/remove fail, causing a leak
[ ] C. ConcurrentModificationException
[ ] D. The key is auto-updated

Why: HashMap never rehashes existing entries. Mutating a key changes its hash, so lookups go to a different bucket and the old entry is stranded.

Q4. Which is the best way to count word frequencies?

[ ] A. map.put(w, map.containsKey(w) ? map.get(w)+1 : 1)
[x] B. map.merge(w, 1, Integer::sum)
[ ] C. map.putIfAbsent(w, 1)
[ ] D. map.compute(w, (k,v) -> v)

Why: merge() handles the missing-key case and the update in one call — concise and atomic on ConcurrentHashMap.

Q5. Why do you get ConcurrentModificationException when removing in a for-each over a map?

[ ] A. The map is immutable
[x] B. The enhanced for uses an iterator, and structural modification invalidates it (fail-fast)
[ ] C. You can't iterate a map
[ ] D. HashMap synchronizes internally

Why: Collection iterators are fail-fast: they track a modCount and throw ConcurrentModificationException when the collection is structurally modified outside the iterator. Use iterator.remove() or removeIf().

Q6. What is the default load factor and why 0.75?

[ ] A. 0.5, to save memory
[x] B. 0.75, a compromise between memory usage and collision frequency
[ ] C. 1.0, to resize as late as possible
[ ] D. 0.9, for speed

Why: 0.75 is the documented trade-off: higher saves memory but increases collisions; lower reduces collisions but wastes space and resizes sooner.

9. Iterators, Comparable vs Comparator Core 30 min

Two small but reliably-asked topics: why removing in a for-each throws, and how sorting actually decides order.

Key points

- **Iterator** works on any Collection. `ListIterator` adds bidirectional traversal, `set()` and `add()`.
- **Fail-fast** iterators (ArrayList, HashMap, HashSet) throw `ConcurrentModificationException` on structural modification during iteration — detected via an internal `modCount`.
- **Fail-safe** iterators (ConcurrentHashMap, CopyOnWriteArrayList) work on a snapshot — no exception, but you may not see live changes.
- **Comparable<T>** — `compareTo(T o)`, defines the *natural* ordering, lives **inside** the class. **Comparator<T>** — `compare(T a, T b)`, an external strategy, defined **outside**.
- Contract: `compareTo` must be consistent with `equals` for sorted collections to behave (return 0 for equal).
- `compare` returns negative / 0 / positive. Never write `return a - b` for ints — it **overflows**; use `Integer.compare(a, b)`.

Reference

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o) — one arg	compare(T a, T b) — two args
Where	Inside the class (this vs o)	Outside, as a separate strategy object
Modifies the class?	Yes	No — good for third-party classes
Used by	Collections.sort(list), TreeSet/TreeMap natural order	Collections.sort(list, cmp), multiple orderings
Number of orderings	One natural ordering	Any number of orderings
Java 8 helpers	-	Comparator.comparing(), thenComparing(), reversed(), nullsLast()

Code

Fail-fast vs fail-safe — see the difference

```
List<String> list = new ArrayList<>(List.of("a","b","c"));

// FAIL-FAST: throws ConcurrentModificationException
// for (String s : list) { if (s.equals("b")) list.remove(s); }

// Correct ways to remove during iteration:
list.removeIf(s -> s.equals("b"));                    // Java 8, cleanest
Iterator<String> it = list.iterator();
while (it.hasNext()) if (it.next().equals("a")) it.remove();
list.removeAll(List.of("a", "b"));                    // bulk removal by value

// FAIL-SAFE: iterates a snapshot, no exception
List<String> cow = new CopyOnWriteArrayList<>(List.of("a","b","c"));
for (String s : cow) { if (s.equals("b")) cow.remove(s); }   // OK

Map<String,Integer> chm = new ConcurrentHashMap<>();
for (String k : chm.keySet()) { chm.put("new", 1); }  // OK - weakly consistent
```

Comparable: natural ordering inside the class

```
class Student implements Comparable<Student> {
   String name; int marks;
   @Override public int compareTo(Student other) {
       return Integer.compare(this.marks, other.marks);   // ascending
       // return Integer.compare(other.marks, this.marks); // descending
   }
}

List<Student> students = new ArrayList<>();
Collections.sort(students);          // uses compareTo
students.sort(null);                 // same thing
// TreeSet<Student> / TreeMap<Student,...> also use it automatically
```

Comparator: external, composable, modern

```
// Old way: an anonymous class
Comparator<Student> byName = new Comparator<>() {
   @Override public int compare(Student a, Student b) {
       return a.name.compareTo(b.name);
   }
};

// Java 8 way - readable and chainable
Comparator<Student> byName   = Comparator.comparing(Student::getName);
Comparator<Student> byMarks  = Comparator.comparingInt(Student::getMarks);
Comparator<Student> combined = byMarks.reversed()          // marks desc
                                      .thenComparing(Student::getName); // then name asc

students.sort(combined);
students.sort(Comparator.comparing(Student::getName, String.CASE_INSENSITIVE_ORDER));

// Null-safe ordering (a real-world must)
students.sort(Comparator.comparing(Student::getName,
             Comparator.nullsLast(String::compareTo)));

// NEVER do this - integer overflow for extreme values:
// (a, b) -> a.getMarks() - b.getMarks()
// DO this:
// (a, b) -> Integer.compare(a.getMarks(), b.getMarks())
```

SAY IT LIKE THIS IN THE INTERVIEW

"Comparable gives a class one natural ordering via compareTo; Comparator is an external strategy via compare, which is better when I need several orderings or can't modify the class. With Java 8 I compose comparators with comparing, thenComparing and reversed. For iteration, collection iterators are fail-fast — structural modification outside the iterator throws ConcurrentModificationException, so I use iterator.remove() or removeIf; concurrent collections are fail-safe instead."

Flashcards

Q. Comparable vs Comparator?
A. Comparable is internal natural ordering (compareTo, one per class). Comparator is an external comparison strategy (compare), allowing many orderings without touching the class.

Q. Fail-fast vs fail-safe iterators?
A. Fail-fast (ArrayList, HashMap) throw ConcurrentModificationException on concurrent structural modification. Fail-safe (ConcurrentHashMap, CopyOnWriteArrayList) iterate a snapshot without throwing.

Q. How do you remove elements while iterating a list?
A. Use iterator.remove(), or the Java 8 removeIf(). Modifying the list directly inside a for-each throws ConcurrentModificationException.

Q. What should compare() return?
A. A negative int if a < b, 0 if equal, positive if a > b — per the agreed ordering.

Q. Why is `a - b` a bad comparator for ints?
A. Integer overflow: for extreme values the subtraction wraps around and returns the wrong sign. Use Integer.compare(a, b).

Q. What does ListIterator add over Iterator?
A. Bidirectional traversal (hasPrevious/previous), plus add() and set() during iteration, and nextIndex/previousIndex.

Q. Should compareTo be consistent with equals?
A. Yes, ideally: compareTo returning 0 should mean equals returns true. Otherwise TreeSet may treat distinct objects as the same element.

Quiz

Q1. Which code throws ConcurrentModificationException?

[ ] A. list.removeIf(x -> x > 2)
[x] B. for (String s : list) list.remove(s)
[ ] C. iterator.remove() inside a while loop
[ ] D. map.entrySet().removeIf(...)

> **Why:** The enhanced for compiles to an Iterator; removing directly from the list bumps modCount and the iterator fails fast. removeIf and iterator.remove() are the safe forms.

Q2. You need to sort Employees by three different fields in different screens. Comparable or Comparator?

[ ] A. Comparable
[x] B. Comparator
[ ] C. Both required
[ ] D. Neither — use SQL

> **Why:** Comparable gives exactly one natural ordering. Multiple orderings are exactly what Comparator is for — define one per screen and pass it to sort().

Q3. Which is the safe way to compare two ints in a comparator?

[ ] A. a - b
[x] B. Integer.compare(a, b)
[ ] C. a < b ? -1 : 1
[ ] D. a == b ? 0 : 1

> **Why:** Integer.compare avoids overflow. (Note option 3 is also overflow-free but loses the three-way contract clarity; Integer.compare is idiomatic.)

Q4. Which collections have fail-safe iterators?

[ ] A. ArrayList and HashMap
[x] B. ConcurrentHashMap and CopyOnWriteArrayList
[ ] C. TreeSet and TreeMap
[ ] D. LinkedList and HashSet

> **Why:** Concurrent collections iterate over a snapshot/weakly consistent view, so concurrent modification doesn't throw.

10. Generics & Wildcards Core 40 min

Generics questions are usually one of two: "explain `? extends` vs `? super`" or "what is type erasure?" Nail PECS and you're covered.

Key points

- Generics give **compile-time type safety** and remove casts: `List<String>` instead of a raw `List`.
- **Type erasure**: the compiler removes generic type info at runtime. `List<String>` and `List<Integer>` are both just `List` to the JVM. That's why you can't do `new T()`, `instanceof List<String>`, or create generic arrays.
- `? extends T` = **upper bound** — T or a subtype. You can **read** as T but not add (producer).
- `? super T` = **lower bound** — T or a supertype. You can **add** a T but reads give Object (consumer).
- **PECS**: Producer `extends`, Consumer `super`. (Joshua Bloch's mnemonic — `Collections.copy(dest, src)` is the canonical example.)
- Bounded type params: `<T extends Comparable<T>>`. Multiple bounds: `<T extends A & B>` (class first, then interfaces).

Code

Why generics exist — the before/after

```
// BEFORE generics (Java 4): unchecked, cast at every use
List names = new ArrayList();
names.add("Parvez");
names.add(42);                          // compiles! runtime bug waiting
String s = (String) names.get(0);       // manual cast, ClassCastException risk

// WITH generics: the compiler catches it
List<String> safe = new ArrayList<>();
safe.add("Parvez");
// safe.add(42);                        // COMPILE ERROR
String s2 = safe.get(0);                // no cast needed

// Generic class
class Box<T> {
   private T value;
   public void set(T v) { this.value = v; }
   public T get() { return value; }
}
Box<Integer> bi = new Box<>();
bi.set(10); int v = bi.get();           // no cast, no boxing surprise

// Generic method (the <T> before the return type is the type parameter list)
static <T> void swap(T[] arr, int i, int j) {
   T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
}
static <T extends Comparable<T>> T max(List<T> list) {
   return list.stream().max(T::compareTo).orElseThrow();
}
```

Wildcards: ? extends vs ? super (PECS)

```
List<? extends Number> producers = new ArrayList<Integer>();
Number n = producers.get(0);        // OK: reading gives at least Number
// producers.add(1);                // COMPILE ERROR: could be a List<Double>!

List<? super Integer> consumers = new ArrayList<Number>();
consumers.add(42);                  // OK: any supertype can hold an Integer
// Integer i = consumers.get(0);    // COMPILE ERROR: only Object is guaranteed
Object o = consumers.get(0);        // OK

// PECS in the wild - Collections.copy:
public static <T> void copy(List<? super T> dest,   // dest CONSUMES T
                            List<? extends T> src) { // src PRODUCES T
   for (int i = 0; i < src.size(); i++) dest.set(i, src.get(i));
}

// Mnemonic: if you GET from it, use extends. If you PUT into it, use super.
// If you do both, use an exact type (no wildcard).

// Unbounded wildcard <?> = "some unknown type" - read as Object, can't add
static void printAll(List<?> list) { for (Object o : list) System.out.println(o); }
```

Type erasure, and what it stops you doing

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());   // true -> both are ArrayList

// Consequences of erasure (all COMPILE ERRORS):
// if (a instanceof List<String>) { }      // can't test a parameterized type
// T obj = new T();                        // can't instantiate a type parameter
// T[] arr = new T[10];                    // can't create a generic array
// class Foo extends Exception<T> { }      // generic throwables not allowed
// void m(List<String> l) & void m(List<Integer> l)  // same erasure -> clash

// What DOES survive: generic info on fields, method signatures and superclasses
// is kept in the class file and readable via reflection (that's how Jackson and
// Spring resolve List<Order> at runtime).

// Workarounds:
static <T> T create(Class<T> clazz) throws Exception {
   return clazz.getDeclaredConstructor().newInstance();   // pass the Class token
}
```

Output:
```
true
```

Raw types and unchecked warnings — what to avoid

```
List raw = new ArrayList();      // raw type: legal for backwards compatibility
raw.add("text"); raw.add(42);    // no type safety at all
List<String> typed = raw;        // unchecked conversion warning

// Mixing raw and generic causes heap pollution:
List<String> strings = new ArrayList<>();
List rawList = strings;
rawList.add(42);                 // warning, but allowed
// String s = strings.get(0);    // ClassCastException at runtime!

// Rules:
//  - Never use raw types in new code.
//  - Prefer List<Object> or List<?> if you truly don't know the element type.
//  - Annotate with @SuppressWarnings("unchecked") ONLY when you can prove safety,
//    and scope it to the smallest local variable possible.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Generics give compile-time type safety and eliminate casts. Java implements them with type erasure — the type parameters are gone at runtime, so List<String> and List<Integer> are the same class, which is why I

can't instantiate a type parameter or test instanceof on a parameterized type. For wildcards I follow PECS: use ? extends when I only read from the structure, ? super when I only write into it."

Flashcards

Q. What is type erasure?
A. The compiler strips generic type information from bytecode: List<String> becomes List at runtime. It enables backwards compatibility but blocks things like `new T()`, generic arrays and `instanceof List<String>`.

Q. What does PECS stand for and mean?
A. Producer extends, Consumer super. Use `? extends T` when the structure only produces (you read) T, and `? super T` when it only consumes (you write) T.

Q. Can you add to a `List<? extends Number>`?
A. No (except null) — the compiler can't know whether the actual list is List<Integer> or List<Double>, so adding could corrupt it.

Q. Can you add to a `List<? super Integer>`?
A. Yes — any supertype of Integer can hold an Integer. But reading only gives you Object.

Q. What is a raw type and why avoid it?
A. A generic type used without type arguments (List instead of List<T>). It exists for pre-Java-5 compatibility, disables type checking and can cause heap pollution.

Q. What is a bounded type parameter?
A. A type parameter with a constraint: `<T extends Comparable<T>>` (upper bound). Multiple bounds use & with the class first: `<T extends Number & Comparable<T>>`.

Q. Can you use primitives as type arguments?
A. No — generics require reference types. List<int> is illegal; use List<Integer> and rely on autoboxing.

Q. Are generic methods possible with static?
A. Yes: `static <T> void m(T t)`. The type parameter is declared on the method, so it's independent of the class's type parameters.

Quiz

Q1. You're writing `static double sum(Collection<? extends Number> nums)`. What does the wildcard buy you?

[ ] A. You can add Integers to the collection
[x] B. You can pass List<Integer>, List<Double> etc. and read them as Number
[ ] C. It makes it faster
[ ] D. It allows null elements

Why: `? extends Number` accepts any collection of a Number subtype and lets you read elements as Number — perfect for a method that only consumes/produces by reading.

Q2. Which signature matches PECS for a method that copies from src into dest?

[ ] A. copy(List<? extends T> dest, List<? super T> src)
[x] B. copy(List<? super T> dest, List<? extends T> src)
[ ] C. copy(List<T> dest, List<T> src)
[ ] D. copy(List<?> dest, List<?> src)

Why: dest receives elements (consumer → super); src provides them (producer → extends). This is exactly how java.util.Collections.copy is declared.

Q3. What is the runtime relationship between List<String> and List<Integer>?

[ ] A. Different classes
[x] B. The same class — generics are erased at runtime
[ ] C. Subtype/supertype
[ ] D. Unrelated but castable

Why: Type erasure means both are `java.util.ArrayList` at runtime; the compiler inserts the casts for you.

Q4. Why can't you write `new T()` inside a generic class?

[ ] A. Constructors can't be generic
[x] B. Erasure means T isn't known at runtime, so there's no constructor to call
[ ] C. T is always an interface
[ ] D. It's a bug in Java

Why: After erasure T is just Object, so the JVM has no idea which constructor to invoke. The standard workaround is passing a `Class<T>` token and using reflection or a Supplier<T>.

Q5. What is the 'diamond' operator <>?

[ ] A. A cast operator
[x] B. Type inference — `new ArrayList<>()` infers the type argument from the context
[ ] C. A wildcard shorthand
[ ] D. Only valid with anonymous classes

Why: Introduced in Java 7, the diamond lets the compiler infer type arguments from the assignment target, removing the duplication of `new ArrayList<String>()`.